

AWS GenAI Developer — Domain 2 (Hinglish)

Domain 1 mein humne dekha ki **kya banao**. Domain 2 mein **kaise banao**. Aur ek badlav — Domain 1 mein maine baar-baar kaha tha "Agents non-deterministic hain, Flows chuno". **Task 2.1 uska doosra pehlu hai:** jab autonomy sach mein chahiye, tab kya karo aur use safe kaise banao.

Task 2.1 — Agentic AI aur Tool Integrations

Use case

Global manufacturing — 50 plants, 15 countries, hazaron components, sainkdon suppliers. Aur yahan naya hai **multi-agent system**.

- Demand forecasting agent
- Inventory optimisation agent
- Supplier management agent
- Logistics coordination agent

Agent hai kya

REASON -> "mujhe inventory data chahiye"
ACT -> inventory API call
OBSERVE -> "stock kam hai, ab supplier dekhna hoga"
REASON -> agla tool ...

Aap raasta nahi batate. Sirf tools dete ho. Yahi Flows se farq hai — Flows mein graph aap banate ho, Agent mein model khud tay karta hai.

Single vs Multi-agent

Multi-agent kyun: demand forecasting statistics hai, supplier management negotiation hai, logistics routing hai.

Ek agent ko yeh sab sikhaoge toh uske **instructions itne bade ho jaayenge ki wo confuse hoga**. Har agent ka apna focus, apne tools, apna prompt.

Service	Kaam
Bedrock Agents	Managed — ek agent, AWS sambhalta hai
Strands Agents	Code-first — custom behaviour, action schemas
Agent Squad	Orchestration — kai agents ko mila kar chalana

Memory hierarchy — teen tiers

- Operational -> abhi ka session -> ElastiCache (sub-ms)
- Tactical -> recent decisions -> DynamoDB
- Strategic -> long-term patterns -> DynamoDB / S3

Operational: agent abhi kya kar raha hai. **Tactical:** pichle hafte ke faisle. **Strategic:** "yeh supplier hamesha late karta hai" — mahinon ka pattern.

Task 1.6 ki DynamoDB conversation history **short-term memory** hi thi. Yahan uske upar do aur layers hain.

ReAct — Step Functions ke saath kyun

Agent khud bhi ReAct karta hai. **Toh Step Functions kyun?**

	Kaun control karta hai
Agent ka apna ReAct	Model — kitne bhi steps, jo bhi order
Step Functions ReAct	Aap — steps defined, par har step ke andar model sochta hai

Step Functions se milta hai: **stopping conditions, state persistence, visibility, auditability**. Yeh beech ka raasta hai — pure Flows (bilkul fixed) aur pure Agents (bilkul free) ke beech.

Safeguards — sabse zaroori hissa

Agent **autonomous** hai. Wo khud faisla leta hai, khud tools chalata hai. **Agar galat raaste par chala jaaye, koi rokne wala nahi.**

```
Agent kuch karna chahta hai
-> [IAM] kya wo yeh kar bhi sakta hai?
-> [Input validation] parameters sahi hain?
-> [Timeout] kitni der laga sakta hai?
-> [Stopping] kitni baar loop kar sakta hai?
-> [Circuit breaker] system healthy hai?
```

Stopping conditions: ReAct loop khud ko rok nahi sakta. Model sochta rahega "thoda aur data chahiye" — infinite. **Agent ko rokne ki zimmedari agent ki nahi, aapki hai.**

IAM sabse important hai: agar inventory-query Lambda ke paas `dynamodb:*` hai, toh agent theoretically **delete** bhi kar sakta hai — chahe aapne wo action define hi na kiya ho. **Action group security boundary nahi hai; IAM hai.**

Analogy: agent ko **chaabiyon ka guccha mat do**. Sirf wo chaabi do jo us kamre ki hai.

Transcript ka asli example: ek bade weather event mein suppliers disrupt huye. System **automatically conservative planning mode** par chala gaya jab tak halat normal na huye. **System band nahi hua — safe mode par gaya.**

Model coordination

```
Claude 3 Sonnet -> complex reasoning
Titan Text     -> report generation
Titan Embeddings -> similarity matching
```

Har kaam ke liye sabse bada model use karna paisa barbaad karna hai. Dhyan dijiye — Titan Embeddings koi "kam" model nahi hai, wo **alag tarah ka** model hai.

Ensemble technique	Kab
Majority voting	Classification, discrete choices
Weighted averaging	Numeric predictions — weights historical performance se

Fallback ke do trigger hain: model FAIL hua, **ya** model ka CONFIDENCE kam hai. Doosra important hai — koi error nahi aaya, par model sure nahi hai.

Human in the loop

Line kahan kheenchein — do factors:

```
confidence > 0.9 -> automatic
0.7 - 0.9        -> AI kare, insaan baad mein dekhe
< 0.7           -> pehle insaan approve kare

Aur RISK se bhi: supplier badalna, production schedule badalna
                  -> hamesha insaan
```

Step Functions task token: workflow **ruka rehta hai**, band nahi hota. State bacha rehta hai — insaan 2 ghante baad approve kare, workflow wahin se chal padta hai.

Human augmentation ka asli matlab: AI **taiyaari** karta hai, insaan **faisla** leta hai. Insaan ka 2 ghante ka kaam 10 minute ka ho jaata hai. **Insaan ko replace nahi, uska time bachao.**

Feedback mein sabse valuable data: agent ne AI ka jawaab badla — matlab galti **aur** sahi jawaab dono mil gaye.

Tool integrations — OpenAPI ka asli role

OpenAPI schema ka description field sabse zyada matter karta hai — kyunki agent yahi padh kar decide karta hai ki kaunsa tool kab call karna hai.

Aam galti: log schema technically sahi likh dete hain par descriptions kharab. Agent phir kaam nahi karta, aur log **model ko blame karte hain**.

Ek line: OpenAPI schema aapka prompt hai, model ke liye.

Error messages model padhega:

```
Bura: "Error 404"
Achha: "Facility PLANT-99 nahi mili. Valid: PLANT-01 se
        PLANT-50. Sahi ID se dobara koshish karo."
```

Doosra message agent ko **theek karne ka mauka** deta hai.

Monitoring se pata chalta hai agent kya soch raha hai: ek tool ka error rate 40% hai — shayad uska **description kharab hai**. Ek tool kabhi call hi nahi hota — shayad agent ko pata hi nahi ki wo hai.

MCP servers — kahan host karo

Host	Kab
Lambda	Stateless, light, <15 min — calculations, lookups, forecasting
ECS	Bhaari, stateful, special environment — simulation engines, code execution

Analogy: Lambda ek **calculator** hai — turant nikaalo, use karo, rakh do. ECS ek **workshop** hai — hamesha khula, saare auzaar rakhe hain.

Tool discovery: agent **poochta hai** ki kya available hai. Naya tool add karo → agent ko apne aap pata chal jaata hai. Powerful, par **predictable kam** — regulated environment mein problem ho sakti hai.

Task 2.2 — Deployment Strategies

Do sawaal

1. Model kahan chalayein? 2. LLM deployment normal ML se alag kyun hai?

Normal ML model	-> 50 MB, CPU par chalta, seconds mein load
LLM	-> 100+ GB, GPU chahiye, MINUTES mein load

Tiered deployment — poore task ka central idea

Tier 1: Simple queries	-> Lambda (lightweight models)
Tier 2: Medium complexity	-> Bedrock (provisioned throughput)
Tier 3: Complex queries	-> SageMaker endpoints (custom models)
^	
Step Functions router + ElastiCache cache	

Har query sabse bade model par nahi jaati. Complexity se route karo, aur jo repeat hota hai use cache karo.

Hosting options

Option	Kab
Bedrock On-Demand	Uneven traffic, commitment nahi. Par shared capacity — throttling ho sakti hai
Bedrock Provisioned	Steady volume. Model Units kharido, har MU guaranteed TPS deta hai
Bedrock Batch	Real-time nahi chahiye — ~50% sasta
Custom Model Import	SageMaker mein train karo, Bedrock serverless pricing par chalao
SageMaker endpoints	Apna model — real-time / serverless / async / batch transform

Lambda mein model host nahi hota. Lambda call karta hai. Timeouts lambe rakho (model seconds leta hai), retry with backoff lagao.

Custom Model Import ka fayda: apna fine-tuned model, par **SageMaker endpoint ka per-hour kharcha nahi**. Exam signal: "custom model + cost optimization".

LLM ki apni dikkatein

Model size:	500 GB tak
Container health check:	60 minutes tak
Model download timeout:	60 minutes tak

Yeh timeouts kyun badhane padte hain: default health check 5-10 minute ka hai. Par 100 GB model download hone mein, phir GPU memory mein load hone mein **20-40 minute** lag sakte hain.

Default settings se deployment FAIL ho jaayegi — SageMaker sochega container dead hai, jabki wo abhi model load kar raha tha. **Yeh aam galti hai.**

Instance selection: har model ko GPU nahi chahiye. NER jaisa chhota model `mL.c5.9xlarge` (CPU) par theek chalta hai. GPU par daaloge toh paisa barbaad.

Model ek GPU mein nahi samaata? GPU mein 80 GB, model 200 GB — **sharding** chahiye. Libraries: Triton, FasterTransformer, DeepSpeed.

Quantization: FP32 (4 bytes) → FP16 (2) → INT8 (1) → INT4. Kam memory, sasta instance, tez inference, **thodi accuracy kam**.

Continuous batching > static batching — GPU khaali nahi baithta.

Optimization

Model cascading: chhote model se shuru, confidence kam ho toh bade par jao.

80% queries chhote model se ho jaati hain. Cost 5x tak kam. Par escalate hone wali queries mein **do calls** lagte hain — latency badhti hai. Isliye cascading tabhi achhi hai jab zyadatar queries sach mein simple hon.

Option	Kya
Multi-model endpoint	Ek instance, kai models. Cold start jab model memory mein nahi
Inference components	Har model ki apni scaling policy — ek ka traffic badha toh sirf wo scale hoga
Serial pipeline	Containers line mein — preprocess → model → postprocess
Multi-container	Kai containers, alag-alag invoke ho sakte hain

Compression ke teen tarike: quantization (precision kam), pruning (weights **hata do**), distillation (bada model chhote ko **sikhata hai**).

Task 2.3 — Enterprise Integration

Problem

Bank, 30 countries. Legacy system decades purana hai, **modern API nahi hai**, SOAP bolta hai. Aur usko badal nahi sakte.

Chetavni: is task mein bahut si services GenAI-specific nahi hain — Transit Gateway, Direct Connect, Network Firewall, Control Tower, WAF. Yeh classic AWS architecture hai GenAI use case ke saath.

Connectivity — legacy ke saamne layer

Legacy XML -> mapping template -> JSON -> Bedrock
Bedrock JSON -> mapping template -> XML/SOAP -> Legacy

Legacy system ko badalna nahi padta — wo wahi XML bhejta hai jo hamesha bhejta tha.

Concurrency setting	Kya karta hai
Reserved	Limit — yeh function itne se zyada nahi chalega
Provisioned	Pre-warmed — cold start nahi , par idle mein bhi bill

Step Functions ke do states: Parallel (alag-alag branches saath) aur **Map** (same kaam, kai items par — batch processing).

Data sync — Glue vs AppFlow

Service	Kab	Khaas feature
Glue	S3, databases, bade ETL	Job bookmarks — yaad rakhta hai kahan tak process kiya, agli baar sirf naya
AppFlow	SaaS apps (Salesforce, ServiceNow)	Ready connectors, bidirectional, filtered

Amazon MQ vs SQS: legacy system pehle se JMS/AMQP bolta hai → **Amazon MQ**. Naya system bana rahe ho → **SQS** (sasta, simple). **MQ sirf isliye hai ki legacy ko badalna na pade.**

Idempotency — yeh sabse important concept hai

Webhook: "loan approved"
-> FM call -> email bheja
Webhook DOBARA aaya (duplicate)
-> FM call -> email DOBARA bheja galat!
-> aur FM call ka paisa do baar laga

External systems at-least-once delivery dete hain. Duplicate aayega hi. Idempotency aapki zimmedari hai — event_id DynamoDB mein store karo (TTL ke saath), pehle dekha ho toh purana result wapas do.

SQS FIFO — message group ID

Customer A ke messages -> group "customer-A"
Customer B ke messages -> group "customer-B"

Har group ke andar order maintain, alag groups parallel. Agar sab ek group mein daal do toh sab sequential — bahut slow. **Business context se group banao** (customer ID) — order bhi mila, parallelism bhi.

Security — saat layers

User login
-> [Federation] enterprise IdP se identity
-> [Cognito trigger] JWT mein enterprise ROLES bhare
-> [WAF] abuse check
-> [PrivateLink] network path + endpoint policy
-> [IAM] AWS-level permission + conditions
-> [Verified Perms] application-level: IS resource par?
-> [KMS] data encrypted
-> [CloudTrail] sab record

IAM vs Verified Permissions:
IAM: "kya aap Bedrock call kar sakte ho?"
Verified Permissions: "**kya aap IS DOCUMENT par call kar sakte ho?**" — classification, department, business context dekh kar.

Cognito pre-token generation trigger: login ke waqt Lambda chalta hai jo JWT mein **enterprise roles** bhar deta hai (department, clearance, region). Ab authorization in claims par ho sakti hai.

WAF rate-based rules GenAI mein khaas: har request paisa hai — abuse = billing attack.

Cross-environment — teen edge options

Service	Kahan	Kyun
Outposts	Aapke data centre	Compliance — data premises se nikal nahi sakta
Local Zones	Sheher mein	Distance — users region se door
Wavelength	Carrier 5G network	Mobile — internet chhoona hi nahi

Direct Connect ka BGP: model inference traffic ko batch processing se zyada priority — same connection, alag priority.

Transit Gateway: dev/test/prod ki alag route tables. Black hole routes — traffic jaan-boojh kar drop, taaki dev se prod tak raasta hi na ho.

Tokenization: real account number ki jagah token. Model token par kaam karta hai — real value kabhi boundary paar nahi karti.

GenAI Gateway — is task ki central architecture

Bina gateway ke: har team apne credentials se Bedrock call kar rahi hai. Kisko pata hai kaun kitna kharch kar raha hai? Guardrails sab jagah lage hain? Audit trail kahan hai?

```
Team A --+
Team B --+--> GenAI Gateway --> Bedrock / SageMaker
Team C --+
```

Gateway deta hai: access control, rate limiting, cost tracking, guardrails jo koi skip nahi kar sakta, model routing, shared caching, audit logging, aur abstraction (model badlo, teams ko farq nahi).

Asli fayda: compliance rule badla → ek jagah badlo, sab jagah lag gaya.

CloudWatch composite alarms: ek metric se pata nahi chalta. Latency zyada — shayad load hai. Par latency aur confidence kam aur error rate zyada — ab pattern hai.

X-Ray sampling: normal requests 5%, slow aur error requests 100%. Detail wahan jahan zaroorat hai.

Task 2.4 — FM API Integrations

2.3 vs 2.4

2.3 -> enterprise ke saath integration (legacy, security, networking)
2.4 -> API level par model se baat karna (timeouts, retries, streaming)

Notes mein bahut **specific numbers** hain. Inhe ratne se zyada zaroori hai **relative logic** — "complex generation ke liye lamba timeout, simple inference ke liye chhota".

Sync ya async — pehla faisla

Pattern	Healthcare example
Synchronous	Clinical query — doctor patient ke saamne khada hai. 4-5 second timeout.
Async (SQS)	Post-encounter documentation — doctor aage badh gaya

Clinical mein 4-5 second kyun: doctor intezaar nahi kar sakta. Agar model 5 second mein nahi de paya, **fallback lena behtar hai** — lamba wait karne se.

Retry aur error handling

Initial backoff: 100 ms
Backoff factor: 2 (100 -> 200 -> 400 -> 800)
Max retries: 3-5
Jitter: +/- 100 ms

Code	Matlab	Karo
429	Throttling	Retry
500 / 503	Server error	Retry
400 / 401 / 403	Bad request / auth	Retry MAT karo

Rule: 4xx **aapki galti** hai — dobara wahi bhejna bekaar. 5xx aur 429 **service ki dikkat** hai — retry se theek ho sakta hai.

SQS visibility timeout processing duration se ZYADA hona chahiye. Kam rakha toh doosra consumer bhi wahi message utha lega — duplicate processing, duplicate cost.

Streaming

Buffer flush kab: buffer bhar gaya, 100-500ms beet gaye, ya **semantic boundary** (sentence poora hua). Teesra sabse behtar hai — aadha vaakya dikhana bura lagta hai.

	WebSocket	SSE
Direction	Bidirectional	Server → client only
Resume	Khud banana padta hai	Built-in (event IDs)

SSE ka event ID resume: connection tooti event #47 par → reconnect → "Last-Event-ID: 47" bhejo → server 48 se aage bhejta hai. **Poora jawaab dobara nahi banana padta.**

LLM streaming ke liye SSE aksar kaafi hai — client ko beech mein kuch bhejna nahi hota.

Stream toot gayi toh: resume try karo → baar-baar fail ho raha hai → **full-response API par fallback.** Poora jawaab ek saath dena aadhe stream se behtar.

Multi-level throttling

Account level: ~10,000 RPS
Stage level: 1,000-5,000 RPS
Route level: 50-500 RPS

Route level sabse important hai — har model ki capacity alag hai. Simple model endpoint 500 RPS chalega, complex 50 se zyada nahi. **Sirf account level throttle karoge toh complex model overwhelm ho jaayega** jabki account limit ke andar hi hai.

Fallback chain — healthcare version

Specialized medical model

- > general model + medical prompts
- > RAG with hospital knowledge base
- > rule-based system

Teesra step RAG hai — model kharab chal raha hai toh hospital ke apne documents se jawaab do. Kam accurate, par **grounded aur safe.**

Monitoring mein p99 kyun: p50 batata hai aam user ka kya haal hai. **p99 batata hai sabse kharab experience kya hai.** Average theek dikh sakta hai jabki 1% users 30 second wait kar rahe hon.

Intelligent routing

Level	Kaise
Static	AppConfig (flags, rollback) ya Parameter Store (simple)
Content-based	Step Functions Choice — tokens, language, classification
Metric-based	DynamoDB (current) ya Timestream (time-series trends)
Cascading	Confidence 0.7-0.9 par escalate

Cascading mein metadata bhi pass karo: specialized model fail hua → general model par jaao → **saath mein pehle model ka context bhi bhejo.** Warna wo shuru se shuru karega.

A/B testing mein statistical significance: 5% traffic par candidate 2% behtar dikha — **kya wo real hai ya sanyog?**

Task 2.5 — Application Patterns aur Dev Tools

Jo naya hai

Yeh task 2.1 aur 2.4 se kaafi overlap karta hai. **Sach mein naya:** token windowing, Amplify UI aur accessibility, Amazon Q Business aur Q Developer, Bedrock Data Automation, prompt registries, synthetic monitoring.

Token windowing aur recursive summarization

```
chunk 1 process      -> summary
chunk 2 + summary    -> process -> nayi summary
chunk 3 + summary    -> ...
```

Har chunk ko pichle ka nichod milta hai — coherence bani rehti hai.

Tiered retry — error type se alag

```
429 (throttle) -> IMMEDIATE retry, jitter 100-300ms
5xx (server)   -> EXPONENTIAL backoff, 500ms se
3-5 failures in 30s -> CIRCUIT BREAK
```

Farq kyun: 429 matlab abhi capacity nahi hai — **thodi der mein mil jaayegi**, isliye turant retry. 5xx matlab kuch toota hai — **zyada time chahiye**.

Accessible interfaces — poora naya hissa

Pattern	Kya
Skeleton screens	Model generate kar raha hai — content ka dhancha dikhao (grey boxes), spinner nahi
Progressive rendering	50-200 ms update interval
Similarity caching	85-95% similar query → cached jawaab. Exact match nahi

ARIA GenAI mein khaas kyun: content **dheere-dheere aata hai**. Screen reader ko batana padta hai ki region update ho raha hai (ARIA live regions).

Prompt Flows non-technical users ke liye: business analysts khud workflows banate hain. Isliye error paths **unhe samajh aane chahiye** — stack trace nahi.

Business system enhancement

CRM enhancement: claim submission ke waqt sentiment analysis → customer record update. **CRM ko badla nahi — bas ek field bhara**.

Q Business	Q Developer
Business users — enterprise search	Developers — code generation, refactoring, testing

Q Business refresh schedules: daily for frequently changing, weekly for stable documentation.

Bedrock Data Automation: unstructured content se structured data — managed. **Task 1.3 se farq:** wahan aapne khud pipeline banayi thi (Textract, Rekognition, Transcribe alag-alag). BDA yeh sab managed deta hai.

Bidirectional sync mein: conflict resolution (AI ne update kiya, insaan ne bhi — kaunsa jeeta?) aur **audit logging of AI-driven changes** — 6 mahine baad pata hona chahiye ki yeh field AI ne badla.

KPI measurement: baseline metrics **AI se PEHLE** establish karo. "Processing 20% tez hui" — **kya wo AI ki wajah se hai ya sanyog?** Baseline ke bina aap sirf daawa kar rahe ho, sabit nahi.

Q Developer — developer productivity

Context dena zaroori hai: project conventions, architectural patterns, dependency versions. Bina context ke Q generic code dega jo codebase se match nahi karega.

Refactoring prioritization: sab kuch refactor nahi kar sakte. **Jo zyada use hota hai aur zyada complex hai, wo pehle**.

AI-assisted testing: coverage code criticality ke hisaab se, aur **GenAI-specific edge cases jaise prompt injection** — yeh normal testing mein nahi aata.

Developer knowledge base: project patterns, decisions, lessons learned → Q ke through available. **Yeh RAG hai, developer knowledge par**.

Advanced applications

Rate limiting agent tools par: agent loop mein fasa toh **bar-bar tool call karega** — resource exhaustion. Task 2.1 ke safeguards mein yeh add hota hai.

Validation between agent handoffs: Agent A ne claim ko "auto collision" samjha, Agent B ne "property damage". **Ab dono sahi kaam kar rahe hain — alag problems par.** Validation step check karta hai ki dono ek hi cheez samajh rahe hain.

Prompt chaining mein context preservation: compression, summarization, ya **key information extraction**. Teesra aksar behtar — summary mein bhi shor hota hai, extraction sirf facts rakhta hai.

Hybrid orchestration: Step Functions batata hai **kya karna hai**, EventBridge batata hai **kab shuru karna hai**.

Troubleshooting

Tool	Kya
Logs Insights	Prompt-response patterns, unexpected token usage (10x tokens = context leak ya loop)
X-Ray annotations	Prompt complexity, token counts — slow requests sab lambe prompts hain jaisi insight
Prompt registry	Latency badhi kal se → kal v8 deploy hua → v8 ka prompt 2x lamba hai → mila

Synthetic monitoring — sabse important naya concept:
Normal monitoring asli traffic dekhta hai — problem tabhi dikhti hai jab user affected ho.
Synthetic khud traffic banata hai — problem **pehle** mil jaati hai. Aur off-peak mein bhi kaam karta hai jab koi user hi nahi hai.

Retrieval results log karna zaroori hai: jawaab galat aaya — **model ki galti thi ya retrieval ki?** Agar retrieved chunks galat the, toh model ne sahi kaam kiya. **Logs ke bina aap galat component tune karte rahoge.**

Domain 2 — Yaad rakhne layak dus baatein

#	Baat
1	Agent ko rokne ki zimmedari aapki hai — ReAct loop khud ko nahi rokta
2	IAM security boundary hai, action group nahi — tool Lambda ke permissions dekho
3	OpenAPI <code>description</code> = agent ka prompt. Kharab description = agent galat tool chunega
4	LLM deployment mein default timeouts se fail hoti hai — 60 min tak badhao
5	Custom Model Import = apna model + serverless pricing
6	Idempotency aapki zimmedari hai — duplicate webhooks aayenge hi
7	SQS FIFO mein message group ID = order + parallelism dono
8	429 → retry turant; 5xx → exponential; 4xx → retry mat karo
9	SSE event IDs se stream resume hota hai — WebSocket mein khud banana padta hai
10	GenAI Gateway = ek jagah access, cost, guardrails, audit — warna sab bikhar jaata hai